12. Technology Stack

The following stack has been selected to optimise for real-time throughput, ML workload flexibility, horizontal scalability, and cost efficiency on the Indian cloud/colocation landscape. Every choice prioritises operational simplicity at startup scale while leaving clear upgrade paths for production growth.

12.1 Infrastructure & Cloud

Component	Technology	Rationale / Purpose
Primary Cloud	AWS (ap-south-1 — Mumbai)	Lowest latency to NSE co-location facility. Data sovereignty compliance. Mature GPU instance types (P3/G5) for ML workloads.
Container Orchestration	Kubernetes (Amazon EKS)	Auto-scaling pattern detection pods during market hours (09:15–15:30 IST). Scale-to-zero overnight for cost optimisation.
CI/CD Pipeline	GitHub Actions + ArgoCD	GitOps workflow with automated staging deploys, canary releases, and automatic rollback on error-rate spike.
Infrastructure as Code	Terraform + Helm Charts	Reproducible infra across dev/staging/prod. All secrets managed via AWS Secrets Manager. No manual console changes.
CDN / Edge	AWS CloudFront + Lambda@Edge	Sub-100ms signal card delivery across India. Static asset caching. API response edge caching for screener pages.
Secrets Management	AWS Secrets Manager + External Secrets Operator	Injects secrets into Kubernetes pods at runtime. Zero hardcoded credentials in code or container images.

12.2 Data Ingestion & Streaming

Component	Technology	Rationale / Purpose
Market Data Feed	NSE Co-location + TrueData / Refinitiv Eikon API	Real-time tick-by-tick OHLCV feed via WebSocket or FIX protocol. Primary + backup vendor for 99.9% uptime.
Message Broker	Apache Kafka (Amazon MSK)	Decouples data ingestion from pattern computation. Handles 2,000 symbols across multiple topics. Replay capability for back-test runs.

Stream Processor	Apache Flink (Kinesis Data Analytics)	Stateful windowed aggregations for OHLCV candle construction (1m, 5m, 15m, 1H, 1D). Exactly-once semantics.
Time-Series Database	TimescaleDB (RDS PostgreSQL 15)	Stores all OHLCV candles with hypertable partitioning by symbol+time. Continuous aggregates for multi-timeframe queries.
Historical Archive	Amazon S3 + Apache Parquet	Cold storage for 5+ years of adjusted OHLCV, partitioned by symbol/year/month for fast back-test scans via Athena.
Low-Latency Cache	Redis 7 (ElastiCache Cluster Mode)	Caches the latest signal card per stock symbol. TTL-based invalidation on each new pattern detection cycle. Also backs Celery broker.
ETL Orchestration	Apache Airflow (MWAA)	Schedules nightly data quality checks, corporate action adjustments, and weekly back-test pipeline runs.

12.3 Pattern Detection Engine

Component	Technology	Rationale / Purpose
Core Language	Python 3.11	Rich ecosystem for numerical computing and ML. Type-annotated with mypy for maintainability and correctness.
Numerical Engine	NumPy + Polars (primary) + Pandas (compat)	Vectorised OHLCV operations. Polars for high-throughput batch scans — 10–50× faster than Pandas for full NSE universe sweeps.
Technical Indicators	TA-Lib 0.4 + pandas-ta	Battle-tested C-backed indicator library covering 150+ functions (RSI, MACD, ATR, Bollinger Bands). pandas-ta for custom indicators.
Pattern ML Layer	scikit-learn + XGBoost 2.0	Gradient boosting models for confidence score calculation. Trained on 5-year labelled historical pattern outcome dataset.
Task Queue / Workers	Celery 5 + Redis Broker	Distributes per-symbol scan jobs across worker pods. Priority queues: Nifty 50 → Nifty 500 → full NSE universe.
Feature Store	Feast (open-source)	Serves pre-computed technical indicator features (RSI, MACD, ATR, volume ratios) to detectors with sub-millisecond latency.

Back-test Analytics	Amazon Redshift Serverless + dbt Core	Weekly back-test pipeline orchestrated by dbt. Pre-computed success rates as materialised views, served via REST API.

12.4 AI Explanation Layer

Component	Technology	Rationale / Purpose
Primary LLM	Anthropic Claude API (claude-3-5-sonnet)	Structured plain-English narrative generation. Superior instruction-following for template-constrained financial output.
Fallback LLM	OpenAI GPT-4o	Automatic failover if primary LLM API is degraded. Identical prompt templates work across both providers.
Prompt Orchestration	LangChain + Jinja2 templates	Manages multi-step prompt chains: pattern data → narrative → disclaimer injection → compliance check. Pydantic output schema enforcement.
Guardrails	Guardrails-AI + custom validators	Blocks recommendation language (buy/sell/hold). Validates all numeric values match source data. Flags SEBI-regulated phrases.
Hindi Translation	IndicTrans2 (AI4Bharat) + Google Translate fallback	High-quality EN→HI translation fine-tuned on financial vocabulary. Google Cloud Translation for edge-case fallback.
LLM Response Cache	Redis (embedding-similarity semantic cache)	Avoids re-generating identical narratives for recurring pattern types on large-cap stocks. Reduces LLM API cost by ~60%.
LLM Quality Evaluation	LangSmith + custom eval harness	Tracks response quality, hallucination rate, and disclaimer compliance. Weekly automated eval + monthly human spot-check.

12.5 Backend API Layer

Component	Technology	Rationale / Purpose
API Framework	FastAPI (Python 3.11) + Uvicorn	Async-first REST + WebSocket API. Auto-generated OpenAPI docs. Sub-10ms framework overhead. 40,000+ req/sec per pod.
API Gateway	AWS API Gateway (HTTP API) + Kong	Rate limiting, JWT auth middleware, request routing, usage analytics, and developer API key management.

Authentication	JWT (RS256) + ET Markets SSO (OAuth 2.0)	Seamless sign-in via existing ET Markets / Times Internet accounts. RBAC for free vs premium feature access tiers.
Real-time Push (Web)	WebSockets (FastAPI native)	Persistent WebSocket connections for live signal card streaming to web users. Heartbeat-based connection health checks.
Mobile Push Notifications	Firebase Cloud Messaging (FCM)	Push alerts to iOS and Android when high-confidence patterns detected on watchlisted stocks.
Service Mesh	AWS App Mesh + Envoy Proxy	mTLS between services, circuit breaking, retries, and traffic shifting for zero-downtime deployments.

12.6 Frontend (Web + Mobile)

Component	Technology	Rationale / Purpose
Web Framework	Next.js 14 (App Router) + TypeScript	SSR for SEO-critical screener pages. React Server Components reduce client JS bundle. ISR for cached signal card pages.
UI Component Library	Tailwind CSS 3 + shadcn/ui	Design-system components. Dark/light mode. WCAG 2.1 AA accessibility. Consistent with existing ET Markets design language.
Charting Engine	TradingView Lightweight Charts v4	Renders interactive OHLCV candlestick charts. Custom SVG annotation layer draws pattern boundaries, targets, and stop zones.
Data Fetching / Cache	TanStack Query (React Query) v5	Server-state caching, background refetch, optimistic updates, and infinite scroll for the screener feed.
Global State	Zustand	Lightweight store for user preferences (timeframe, threshold, language) and watchlist optimistic state.
Mobile App	React Native (Expo SDK 51)	Shared business logic with web. Native push notifications via Expo Notifications. Single codebase for iOS and Android.
E2E Testing	Playwright (web) + Detox (mobile)	Automated end-to-end regression tests covering the screener, signal card rendering, and push notification flows.

12.7 Observability, Security & MLOps

Component	Technology	Rationale / Purpose

Application Monitoring	Datadog APM + Distributed Tracing	Full trace from user request through pattern engine to LLM response. P50/P95/P99 latency dashboards per endpoint.
Log Aggregation	OpenSearch (managed ELK)	Centralised structured JSON logging. Pattern detection audit trail. LLM prompt/response logs for compliance review.
Infrastructure Metrics	Prometheus + Grafana	Kubernetes cluster metrics, Kafka consumer lag, Redis memory, TimescaleDB query performance — all on one pane of glass.
Alerting & On-Call	PagerDuty + Datadog Monitors	Escalation policies for: data feed lag > 2 min, pattern engine error rate > 1%, API P99 > 3s, LLM guardrail violations.
ML Model Registry	MLflow + S3 Model Store	Versioned confidence score models. A/B testing framework for new candidates. Automated weekly retraining on fresh outcome data.
SAST / Dependency Scan	GitHub CodeQL + Snyk	Static analysis on every PR. Automated CVE scanning of Python and Node.js dependencies. Blocks merges with critical vulnerabilities.
Data Encryption	AWS KMS + TLS 1.3 everywhere	All data encrypted at rest (KMS) and in transit (TLS 1.3). TimescaleDB and S3 buckets use customer-managed keys.

13. Complete Project Folder Structure

The codebase is organised as a Turborepo monorepo with PNPM workspaces for JavaScript packages and Python virtual environments per service. Every service is independently deployable via its own Dockerfile and Helm chart. Shared TypeScript types and React components are published as internal workspace packages.

13.1 Root Monorepo

Path	Description
`ai-indian-investor/`	Monorepo root — git repository root
`├── apps/`	All end-user–facing deployable applications
`│  ├── web/`	Next.js 14 web application (ET Markets frontend integration)
`│  ├── mobile/`	React Native (Expo) iOS + Android app

│ └── admin/	Internal ops dashboard for monitoring pattern engine health
├── services/	Backend microservices — each independently containerised
│ ├── data-ingestion/	NSE market data feed consumer and OHLCV normaliser
│ ├── pattern-engine/	Core pattern detection computation service (Python / Celery)
│ ├── backtest-runner/	Historical back-test pipeline and analytics store loader
│ ├── ai-explanation/	LLM orchestration and plain-English signal explanation service
│ ├── signal-api/	Public REST + WebSocket API gateway (FastAPI)
│ └── notification/	Push notification and email digest delivery service
├── packages/	Shared internal packages (referenced via pnpm workspace:*)
│ ├── types/	Shared TypeScript type definitions: Pattern, SignalCard, Stock, User
│ ├── ui/	Shared React component library: SignalCard, ChartWidget, ConfidenceBadge
│ ├── config/	Shared ESLint, Prettier, Tailwind CSS, tsconfig, and pytest presets
│ └── utils/	Shared Python + TypeScript utility functions (formatters, validators)
├── infra/	All cloud infrastructure definitions
│ ├── terraform/	AWS resource provisioning (EKS, RDS, MSK, ElastiCache, S3, CloudFront)
│ ├── helm/	Helm charts for all Kubernetes service deployments
│ └── scripts/	One-off ops scripts: DB migrations, data backfills, index rebuilds
├── data/	Data pipeline definitions and transformation models
│ ├── dbt/	dbt project for back-test analytics on Amazon Redshift
│ ├── migrations/	TimescaleDB schema migrations managed by Alembic
│ └── seeds/	Reference data: NSE symbol master, sector/industry mappings, holiday calendar
├── ml/	Machine learning model training, evaluation, and experiment tracking
│ ├── experiments/	Jupyter notebooks and MLflow experiment configurations

│ ├── models/	Trained model artefacts, configs, and feature importance reports
│ └── eval/	Back-test accuracy evaluation harnesses and labelled fixture datasets
├── docs/	Living project documentation
│ ├── adr/	Architecture Decision Records (numbered, dated, immutable)
│ ├── api/	OpenAPI 3.1 specs for all services (auto-generated + hand-written)
│ └── runbooks/	Ops runbooks for incident response, on-call procedures, and game days
├── .github/	GitHub configuration
│ ├── workflows/	CI/CD workflows: per-service build, test, Docker publish, Helm deploy
│ └── CODEOWNERS	Auto-assign reviewers by directory ownership
├── turbo.json	Turborepo pipeline: build → test → lint → deploy dependency graph
├── pnpm-workspace.yaml	PNPM workspace definition — links all JS apps and packages
├── pyproject.toml	Root Python tooling config (black, isort, mypy, pytest settings)
└── Makefile	Top-level developer commands: make dev, make test, make deploy-staging

13.2 services/data-ingestion/

Path	Description
data-ingestion/	
├── src/	
│ ├── feeds/	Pluggable feed adaptor implementations
│ │ ├── base_feed.py	Abstract base class defining feed adaptor interface
│ │ ├── nse_websocket.py	NSE real-time WebSocket feed consumer (primary)
│ │ ├── truedata_feed.py	TrueData vendor REST/WebSocket API adaptor
│ │ └── refinitiv_feed.py	Refinitiv Eikon API adaptor (backup feed)

│ ├── normaliser/	Data cleaning, validation, and standardisation
│ │ ├── ohlcv_normaliser.py	Validates tick data, fills missing fields, deduplicates
│ │ └── corporate_actions.py	Applies splits, bonuses, dividends to adjust price history
│ ├── candle_builder/	Constructs completed OHLCV candles from raw ticks
│ │ ├── candle_builder.py	Aggregates ticks into 1m / 5m / 15m / 1H / 1D candles
│ │ └── timeframe_config.py	Timeframe definitions, NSE market hours, holiday calendar
│ ├── publishers/	Writes processed candles to downstream sinks
│ │ ├── kafka_publisher.py	Publishes completed candle events to Kafka topics by timeframe
│ │ └── timescale_writer.py	Bulk-writes completed candles to TimescaleDB
│ ├── health.py	HTTP /health endpoint for Kubernetes liveness probe
│ └── main.py	Service entry point — wires feed → normaliser → candle builder → publishers
├── tests/	
│ ├── unit/	Unit tests for normaliser, candle builder, and corporate actions logic
│ └── integration/	Integration tests against a local Kafka + TimescaleDB test environment
├── config/	Environment-specific YAML config files (dev / staging / prod)
├── Dockerfile	Multi-stage production Docker image (python:3.11-slim base)
└── requirements.txt	Pinned Python dependencies with hashes

13.3 services/pattern-engine/

Path	Description
pattern-engine/	
├── src/	
│ ├── detectors/	One module per pattern family — all implement BaseDetector interface
│ │ ├── base_detector.py	Abstract class: detect(ohlcv: DataFrame) -> list[PatternResult]
│ │ ├── breakouts/	

`|  |  |  ├── cup_and_handle.py`	Cup & Handle formation detection with handle tightness validation
`|  |  |  ├── ascending_triangle.py`	Ascending triangle with flat resistance + rising support
`|  |  |  ├── rectangle_breakout.py`	Rectangle consolidation breakout (both directions)
`|  |  |  └── vcp.py`	Volatility Contraction Pattern (Mark Minervini method)
`|  |  ├── reversals/`	
`|  |  |  ├── head_and_shoulders.py`	H&S Top and Inverse H&S Bottom — neckline break confirmation
`|  |  |  ├── double_top_bottom.py`	Double Top and Double Bottom with retest validation
`|  |  |  └── triple_top_bottom.py`	Triple Top and Triple Bottom detector
`|  |  ├── candlesticks/`	
`|  |  |  ├── single_candle.py`	Doji, Hammer, Shooting Star, Spinning Top, Marubozu
`|  |  |  └── multi_candle.py`	Engulfing (Bull/Bear), Morning/Evening Star, Harami, Three Soldiers/Crows
`|  |  ├── divergences/`	
`|  |  |  ├── rsi_divergence.py`	Regular and Hidden Bullish/Bearish RSI divergence
`|  |  |  └── macd_divergence.py`	MACD histogram and signal-line divergence detector
`|  |  └── momentum/`	
`|  |  ├── ma_crossover.py`	Golden Cross, Death Cross, EMA 9/21 and 50/200 crossovers
`|  |  └── support_resistance.py`	Dynamic (EMA-based) and horizontal (pivot) S/R level detection
`|  ├── confidence/`	Confidence score engine
`|  |  ├── scorer.py`	Composite 0–100 score: pattern quality + volume + market breadth + sector
`|  |  ├── features.py`	Feature vector extraction for the XGBoost confidence model
`|  |  └── model_loader.py`	Loads trained XGBoost model artefact from MLflow model registry
`|  ├── indicators/`	Shared indicator computations
`|  |  ├── ta_wrapper.py`	Thin wrapper around TA-Lib for consistent OHLCV API

│ │ └── feature_fetcher.py	Fetches pre-computed features from Feast feature store
│ ├── scanner/	Universe scanning orchestration
│ │ ├── universe_scanner.py	Full NSE universe scan loop dispatching Celery tasks per symbol
│ │ ├── priority_queue.py	Scan priority: Nifty 50 (1m) → Nifty 500 (5m) → all NSE (15m)
│ │ └── result_publisher.py	Writes detected PatternResult objects to Redis + Kafka pattern-events topic
│ └── main.py	Celery worker entry point and beat scheduler registration
├── tests/	
│ ├── fixtures/	Labelled historical OHLCV snapshots with ground-truth pattern annotations
│ ├── unit/	Unit tests for each detector (precision + recall against fixtures)
│ └── integration/	End-to-end scanner tests using a local TimescaleDB + Redis stack
├── notebooks/	Jupyter notebooks for detector parameter tuning and visualisation
├── Dockerfile	
└── requirements.txt	

13.4 services/ai-explanation/

Path	Description
ai-explanation/	
├── src/	
│ ├── chains/	LangChain prompt chain definitions
│ │ ├── signal_chain.py	Main chain: pattern data → narrative → disclaimer injection → final card
│ │ └── quality_chain.py	Secondary validation chain — checks output for compliance violations
│ ├── prompts/	Jinja2 prompt template library
│ │ ├── signal_narrative.j2	English signal narrative template with structured slots for all numeric values

│ │ ├── hindi_translate.j2	Translation prompt with domain-specific financial vocabulary hints
│ │ └── disclaimer.txt	Static SEBI-compliant disclaimer appended to every signal card
│ ├── guardrails/	Output validation and compliance enforcement
│ │ ├── output_schema.py	Pydantic V2 schema defining required fields and type constraints for all LLM output
│ │ ├── recommendation_filter.py	Regex + NLP filter that blocks any buy / sell / hold recommendation language
│ │ └── numeric_grounding.py	Verifies every price level and percentage in LLM output matches the source data payload
│ ├── cache/	Semantic caching layer
│ │ ├── semantic_cache.py	Embedding-similarity cache using Redis + Sentence Transformers to skip re-generation
│ │ └── cache_warmer.py	Pre-warms cache at market open for top 50 most-watched stocks
│ ├── translation/	Hindi translation pipeline
│ │ ├── indicTrans_client.py	IndicTrans2 model inference client (self-hosted on EKS GPU node)
│ │ └── google_translate_fallback.py	Google Cloud Translation API fallback with financial glossary overrides
│ ├── providers/	LLM provider abstraction
│ │ ├── base_provider.py	Abstract LLM provider interface
│ │ ├── anthropic_provider.py	Anthropic Claude API client with retry, timeout, and cost tracking
│ │ └── openai_provider.py	OpenAI GPT-4o fallback provider
│ └── main.py	FastAPI service exposing POST /explain endpoint consumed by signal-api
├── evals/	LangSmith evaluation harness
│ ├── eval_dataset.jsonl	Gold-standard input/output pairs for automated quality scoring
│ └── run_evals.py	Weekly eval runner: accuracy, hallucination rate, disclaimer compliance metrics
├── Dockerfile	

└── requirements.txt	

13.5 services/signal-api/

Path	Description
signal-api/	
├── src/	
│ ├── routers/	FastAPI route handlers (one file per resource)
│ │ ├── signals.py	GET /signals · GET /signals/{symbol} · GET /signals/screener
│ │ ├── watchlist.py	GET/POST/DELETE /watchlist — user watchlist CRUD
│ │ ├── backtest.py	GET /backtest/{symbol}/{pattern} — historical success stats
│ │ ├── preferences.py	GET/PUT /user/preferences — timeframe, confidence threshold, language
│ │ └── ws.py	WebSocket /ws/signals — real-time signal streaming to web clients
│ ├── models/	Database ORM models and Pydantic API schemas
│ │ ├── signal.py	SignalCard ORM model + REST response schema
│ │ ├── user_prefs.py	UserPreferences ORM model
│ │ └── watchlist.py	Watchlist ORM model with user FK
│ ├── services/	Business logic layer — orchestrates DB, cache, and downstream services
│ │ ├── signal_service.py	Fetches signals from Redis (hot) / TimescaleDB (warm), applies user filters
│ │ ├── screener_service.py	NSE screener: filter by pattern/sector/confidence, sort, paginate
│ │ └── backtest_service.py	Queries Redshift back-test success rate materialised views
│ ├── middleware/	
│ │ ├── auth.py	JWT RS256 validation + ET Markets SSO OAuth 2.0 token exchange
│ │ ├── rate_limiter.py	Per-user sliding window rate limiting backed by Redis
│ │ └── premium_gate.py	Feature-level access control: free tier vs premium tier

`|  ├── db/`	Database connection and session management
`|  |  ├── timescale.py`	TimescaleDB async SQLAlchemy connection pool
`|  |  └── redis_client.py`	Shared Redis connection pool
`|  └── main.py`	FastAPI app init, router registration, lifespan hooks, CORS config
`├── tests/`	
`|  ├── unit/`	Service layer unit tests with mocked dependencies
`|  └── e2e/`	End-to-end API tests using httpx against a local stack
`├── Dockerfile`	
`└── requirements.txt`	

13.6 apps/web/ (Next.js Frontend)

Path	Description
`apps/web/`	
`├── app/`	Next.js 14 App Router — directory = route segment
`|  ├── (market)/`	Route group: market-facing pages (no layout segment in URL)
`|  |  ├── screener/`	Pattern screener hub
`|  |  |  ├── page.tsx`	Full NSE screener with pattern / sector / confidence / timeframe filters
`|  |  |  └── loading.tsx`	Skeleton loader shown during RSC streaming
`|  |  ├── stock/[symbol]/`	Dynamic stock detail page
`|  |  |  ├── page.tsx`	Stock overview with active pattern card at top of technical section
`|  |  |  └── patterns/`	Pattern history sub-route for this stock
`|  |  |  |  └── page.tsx`	Full historical pattern list with back-test stats for symbol
`|  |  └── watchlist/`	User watchlist with live pattern alert badges
`|  |  |  └── page.tsx`	
`|  ├── api/`	Next.js Route Handlers — Backend-for-Frontend (BFF) layer

`|  |  └── signals/`	Proxies signal-api with server-side session auth — no API key exposed to client
`|  ├── layout.tsx`	Root layout: AuthProvider, ThemeProvider, QueryClientProvider
`|  └── globals.css`	Tailwind base styles and CSS custom properties
`├── components/`	Reusable React components
`|  ├── signal-card/`	Signal card component family
`|  |  ├── SignalCard.tsx`	Full card: pattern name, narrative, confidence, action zone, back-test stats
`|  |  ├── SignalCardMini.tsx`	Compact card for screener list view (symbol + pattern + confidence only)
`|  |  ├── ConfidenceBadge.tsx`	Colour-coded badge: 0–50 red, 51–70 amber, 71–100 green
`|  |  └── DisclaimerFooter.tsx`	SEBI disclaimer rendered on every signal card — cannot be hidden
`|  ├── chart/`	Charting components
`|  |  ├── CandlestickChart.tsx`	TradingView Lightweight Charts v4 wrapper with SSR guard
`|  |  └── PatternOverlay.tsx`	SVG canvas layer rendering pattern boundaries, necklines, entry/target/stop zones
`|  ├── screener/`	Screener page components
`|  |  ├── FilterPanel.tsx`	Left sidebar: pattern type tree, sector multi-select, confidence slider, timeframe toggle
`|  |  ├── SignalTable.tsx`	Paginated, sortable signal results table with infinite scroll
`|  |  └── MarketPulseWidget.tsx`	Homepage widget: top 5 highest-confidence patterns of the day
`|  └── notifications/`	
`|  |  ├── NotificationCentre.tsx`	In-app notification feed with mark-read and filter
`|  |  └── AlertSetupModal.tsx`	Per-stock alert configuration: confidence threshold + pattern types
`├── lib/`	Frontend utilities
`|  ├── api-client.ts`	Type-safe React Query hooks wrapping all signal-api endpoints
`|  ├── ws-client.ts`	WebSocket client class with auto-reconnect and backoff

│ └── formatters.ts	Price (₹), percentage, date, large-number (₹ crore/lakh) formatters
├── store/	Zustand global state stores
│ ├── userPrefsStore.ts	Timeframe, confidence threshold, language preference — persisted to localStorage
│ └── notificationStore.ts	Unread notification count and notification list state
├── next.config.ts	Next.js config: output standalone, image domains, CSP headers
├── tailwind.config.ts	Tailwind theme: ET Markets brand colours, typography scale
└── tsconfig.json	TypeScript config — extends shared @repo/config/tsconfig.json

13.7 infra/terraform/ (AWS Infrastructure)

Path	Description
infra/terraform/	
├── modules/	Reusable Terraform modules — each manages one AWS resource group
│ ├── eks/	EKS cluster, managed node groups, IRSA IAM roles, cluster autoscaler
│ ├── rds-timescale/	RDS PostgreSQL 15 + TimescaleDB extension, Multi-AZ, automated backups
│ ├── elasticache/	Redis 7 cluster mode, encryption at rest, automatic failover
│ ├── msk/	Amazon MSK (Managed Kafka 3.6), 3-broker cluster, TLS encryption
│ ├── redshift/	Redshift Serverless workgroup for back-test analytics
│ ├── s3/	S3 buckets: historical-data (Parquet), model-artefacts, terraform-state
│ ├── cloudfront/	CloudFront distribution: API caching + static asset CDN + WAF
│ ├── airflow/	Amazon MWAA (Managed Airflow) environment for ETL orchestration
│ └── secrets/	AWS Secrets Manager secrets + External Secrets Operator config
├── environments/	Per-environment variable overrides

`|  ├── staging/`	Staging: smaller instances, single-AZ, reduced Kafka replication factor
`|  |  ├── main.tf`	Calls all modules with staging variable values
`|  |  └── terraform.tfvars`	Staging-specific variable values (instance sizes, replica counts)
`|  └── production/`	Production: multi-AZ, auto-scaling, cross-region S3 replication
`|  ├── main.tf`	
`|  └── terraform.tfvars`	
`├── main.tf`	Root module: VPC, subnets, security groups, wires all submodules
`├── variables.tf`	Input variable declarations with descriptions and validation rules
`└── outputs.tf`	Exports cluster endpoint, DB connection string, Redis endpoint for Helm values

13.8 data/dbt/ (Back-test Analytics Pipeline)

Path	Description
`data/dbt/`	
`├── models/`	dbt SQL models — organised in three layers
`|  ├── staging/`	Raw source materialisation — one model per source
`|  |  ├── stg_ohlcv.sql`	Stages OHLCV data from S3 Parquet (via Redshift Spectrum)
`|  |  └── stg_patterns.sql`	Stages detected pattern events from Kafka → S3 archive
`|  ├── intermediate/`	Business logic — pattern outcome labelling
`|  |  ├── int_forward_returns.sql`	Computes 5/10/20-day forward returns for every pattern event
`|  |  └── int_pattern_outcomes.sql`	Labels each pattern occurrence: success (return > threshold) or failure
`|  └── marts/`	Final analytical tables — consumed by signal-api backtest endpoint
`|  ├── backtest_success_rates.sql`	Win rate per (symbol × pattern_type × timeframe) — materialised weekly

`|  ├── avg_pattern_gain.sql`	Average return and max drawdown per winning trade instance
`|  └── pattern_stats_summary.sql`	Aggregate stats used in screener sort-by-success-rate filter
`├── tests/`	dbt data quality tests: not_null, unique, accepted_values, referential integrity
`├── macros/`	Reusable dbt Jinja macros: date_spine, forward_return, pattern_label
`├── seeds/`	CSV seed files: pattern_type_definitions, sector_master, nifty_constituents
`├── snapshots/`	dbt snapshots: tracks changes in NSE symbol master over time
`└── dbt_project.yml`	dbt project config: materialisation strategy, target schema, var definitions

13.9 ml/ (Machine Learning)

Path	Description
`ml/`	
`├── experiments/`	MLflow experiments — each subdirectory is one model family
`|  ├── confidence_score/`	XGBoost confidence score model training and hyperparam tuning
`|  |  ├── train.py`	Training script: loads feature store data, trains, logs to MLflow
`|  |  ├── evaluate.py`	Evaluation: precision@k, rank correlation, calibration curve
`|  |  └── hyperparameter_search.py`	Optuna-based Bayesian hyperparam search
`|  └── pattern_classifier/`	Optional: CNN/LSTM model for visual chart pattern classification
`├── features/`	Feature engineering pipeline
`|  ├── feature_definitions.py`	Feast feature view definitions for all technical indicators
`|  └── feature_pipeline.py`	Airflow DAG that computes and ingests features into Feast
`├── models/`	Production model artefacts (synced from MLflow model registry)
`|  └── confidence_v2/`	Current production confidence score model weights + config

`└── eval/`	Accuracy evaluation framework
`  ├── labelled_fixtures/`	Manually labelled OHLCV snapshots with verified pattern annotations
`  └── accuracy_report.py`	Generates weekly detector precision / recall report published to Datadog

Appendix — Glossary of Terms

Term	Definition
OHLCV	Open, High, Low, Close, Volume — the five data points recorded for each time period of a security's price history.
NSE	National Stock Exchange of India — the largest stock exchange in India by daily trading volume.
Demat Account	Dematerialised account — a digital account that holds shares and securities in electronic form.
RSI	Relative Strength Index — a momentum oscillator measuring the speed and change of price movements on a 0–100 scale.
MACD	Moving Average Convergence Divergence — a trend-following momentum indicator showing the relationship between two exponential moving averages.
ATR	Average True Range — a measure of market volatility used to size stop-loss distances and position sizes proportionally.
SIP	Systematic Investment Plan — a method of investing a fixed amount at regular intervals in mutual funds.
F&O	Futures and Options — exchange-traded derivative instruments available on select NSE equities and indices.
Back-test	The process of testing a trading strategy or pattern detection rule against historical data to estimate its likely future performance.
SEBI IA Regs	Securities and Exchange Board of India (Investment Advisers) Regulations, 2013 — the regulatory framework governing who can provide investment advice in India.
Monorepo	A version control strategy where all services and packages for a project live in a single repository, enabling shared tooling and atomic cross-service changes.
Turborepo	An open-source high-performance build system for JavaScript/TypeScript monorepos that intelligently caches and parallelises build tasks.

dbt	Data Build Tool — an open-source framework for defining, testing, and documenting SQL-based data transformation pipelines.
MLflow	An open-source platform for managing the full machine learning lifecycle, including experiment tracking, model versioning, and deployment.
Feast	An open-source feature store that manages the definition, storage, and serving of ML features for both training and online inference.

— End of Document —

AI for the Indian Investor · ET Markets · Times Internet Limited · Version 1.1 · March 2026